

面试总结

一、荣耀相机外包（一面）

中科创达 荣耀外包 侯晴 11-18k（我没给他报） 相机开发 3月3号下午4点

1、自我接受

我想应聘的是一个 C++开发岗，注重细节，对于写代码我的态度比较严谨，喜欢深思熟虑后再输出，注重代码的格式和可读性。熟悉 C++11的相关特性，然后熟悉STL、socket、epoll的使用然后除了技术，我觉得一个项目要写好，需要团队有好的配合，不能只顾着研究自己的。总的来说，我喜欢C++开发，希望有机会能加入贵公司，共同成长。

2、离职原因?

原来公司的核心项目调整，转向了在原有的框架结构上做一些简单增删改查类需求，我更希望参与到技术驱动型项目。我更希望能够找到一个能持续学习和进步的机会。

3、项目内容

它里边儿有秒传功能，然后创建MySQL的任务，然后根据文件的希值来查，看那个服务器这边到底有没有上传过这个文件，如果已经上传过这个文件，直接就返回 success，直接就上传成功，然后还有断点续传，上传的时候会没有上传成功的话会有一个临时的文件夹，然后寻找是否有同名的文件夹，如果有的话查看 upload ID的这个表上看哪个标志位是0，如果这个标志位是0的话，就上传对应的分片。

在传输系统中做了秒传、断点续传、分片上传等。在搜索与查询系统主要做了构建倒排索引和缓存优化，做查询相关的性能优化的时候，发现在算法上优化很难，所以想到的是在框架结构上去优化。想要进行优化的话，就要先确定性能瓶颈到底是在哪。需要做压力测试（profile benchmark），生成火焰图，寻找最大宽度。确定瓶颈是在磁盘IO，于是想到了要做缓存。为了能对性能优化，开始使用的是直接用Redis做缓存。有了缓存系统后，效率的到了大幅度的提升。但还没达到理想的状态。查询

Redis的话大概是**0.1毫秒**，直接查看主存的话大概是**0.1微妙**，速度差距还是很大的。所以做一个LRU实现自定义缓存，会极大的提升效率。Redis自带**原子性**，不需要考虑加锁解锁的情况，自己做公共的LRU缓存就需要**加锁**。但是加锁又会损失性能，并不是说加锁解锁本身会影响性能，而是取决于**临界区的长度**。所以要思考如何减少临界区长度。最初想到的是用多缓存，**每个线程有自己的LRU缓存**，这样就不用加锁了，但是会造成**命中率下降**，甚至比加锁性能损耗的更严重。为了提高命中率，引入了公共缓存，**每个线程自己的缓存定时的与公共缓存进行同步**。只有同步的时候加锁，这样就临界区和命中率比较平衡了。想法是好的，实际表现和只有公共缓存的差不多。最后的解决方案是使用双缓存，使更新和同步分离。整体思路是**每个工作线程都有两个LRU缓存**，一个用于执行任务，一个用于与公共缓存同步，这样的话只有在内部两个缓存进行同步的时候才会加锁，进一步减少访问临界区的时间，整体的效率比单独使用Redis提升了**20%~30%**。

注：如果面试官想听再多可以再提：秒传、断点续传、分块传输。

断点续传：先检查寻找是否存在同名 `uploadid_tmp_dir`，①存在就读取 Redis 的 `uploadid` 表 记录所有 `chunknum`标志位=0 【等于重新传输该分片】②响应JSON (`uploadid: {chunknum1, chunknum2}`)；③所有分片上传完成后，服务端将 `uploadid_tmp_dir` 中的分片按顺序合并为完整文件。⑤删除 `uploadid_tmp_dir` 并清理 Redis 中的 `uploadid` 表。

4、它这个表的话，它是每上传一个文件它都会创建吗？

主要是大文件，小文件的话跟大文件的传输不太一样，他会先判断这个文件的大小，如果文件比较小的话直接就传，如果是文件比较大的话再进行分片。

5、那文件小如果传输失败会怎么样？

传输失败的话会重新传。

6、你怎么判断它的大小？

通过服务端的API网关判断文件的大小，文件比较小的话就直接recv就接收，文件比较大的话就先分片，分片了之后再用mmap和 recv在接受。

7、对算法这边接触的多吗，类似实现算法移植这些东西？

使用过第三方库，根据那些文档，然后如何使用，都是用过的。

8、有三方厂家给的SDK文档是吧？

对。

9、你这个项目当时是在PC端的项目，有没有做过嵌入式的开发？

没有做过嵌入式开发。

10、对交叉编译工具链了解的多吗？

没有了解

11、const 和 static

static是静态的，加上static之后，它就会变成一个全局的，然后它在要在类里边声明，在类外定义，可以全局使用。

const修饰 数据成员：在初始化列表中初始化、一经初始化就不可更改。

const修饰 成员函数：其内部不可修改成员、常用作查看数据

static修饰 数据成员：类内定义，类外初始化。不占对象空间，存储在全局静态区，所有该类对象共享。

static修饰 成员函数：依附于类 没有 this 指针 只能访问静态的数据成员

12、面向对象语言的三大特征？

继承、多态、封装、抽象

13、多态的实现原理

多态有静态多态和动态多态，然后静态多态是编译时多态，有函数名重载，还有运算符重载，然后动态多态主要是通过虚函数实现，函数重载的话是在同一个作用域当中，它函数名相同，然后参数不同，就能构成函数重载，运算符重载的话是自定义操作符的运算规则。

静态多态（编译时多态）：

实现方式两种：

函数名重载：同一作用域中，函数名相同，只有参数列表不同

运算符重载：允许为自定义类型（如类或结构体）自定义运算符的规则。

动态多态（运行时多态）：虚函数实现

虚函数定义：基类中virtual修饰的成员函数、基类中声明，派生类中定义，实现覆盖。返回参数、函数名、参数列表都相同，（要求的格式是最严格的），通常加上关键字 `override` 来保证虚函数的书写是正确的

实现动态多态的5个条件：

- 1、基类中有虚函数；
- 2、派生类中有同名的函数，实现覆盖（重写）；
- 3、创建派生类对象；
- 4、基类指针指向派生类对象；
- 5、基类指针调用虚函数；

场景	虚表数量
无虚函数	0
单继承	1（基类的虚表扩展而来）
多继承	等于基类数量（每个基类贡献一个虚表）
虚继承	每个有虚函数的基类对应一个虚表 + 虚基类管理开销（具体由编译器实现决定）

抽象类，只定义，由派生类实现

14、strcpy和mmap

这里的问题是strcpy和memcpy的区别是什么？

mmap和直接从磁盘直接传输到网络，它不需要经过内核态和用户态之间的多重拷贝会更快一点，strcpy会经过内核态和用户态的之间的那些口碑也会速度慢一点。

strcpy是C 标准库函数，用于复制字符数组。仅操作内存中的字符数组，不涉及系统资源（如文件、硬件等）。

mmap是内存映射：系统调用（Unix/Linux），用于将文件、设备或匿名内存映射到进程的虚拟地址空间。用于大文件处理、共享内存等。

15、程序内存分配

16、它的栈区和堆区主要存那些东西？

栈区存函数调用栈、自动管理，堆区存自己申请的空间，要注意释放申请的堆空间。

程序内存布局

这次面试给人的感觉是基础不扎实，回答的很生涩，八股文还是背诵得少了。连最简单的static关键字的用法都没有说的很清楚，这是肯定不太合适的。

程序内存布局，这种问题也是很基础的。六个区域：内核区、栈区、堆区、全局静态区、文字常量区、程序代码区。这些都是要做到随口就可以说的

自己主动提问中，特别提了周末加班多吗？这个是不适合的，别人还没有提这点儿的时候，我们不需要主动提了，提了别人可能就会认为你不愿意加班的。

这场面试只有13分钟就结束了，时间太短，了解得也不够多。

内存分
区 说明

由下到
上

程序代

码区 存放函数体的二进制代码。一个C语言程序由多个函数构成，C语言程序的执
(code 行就是函数之间的相互调用。这块内存只有读取权限。
)

常量区 存放一般的常量、字符串常量等。这块内存只有读取权限，没有写入权限，
(const
ant) 因此他们的值在程序运行期间不能改变。

全局数

据区 存放全局变量、静态变量等。这块内存有读写权限，因此他们的值在程序运
(globa 行期间可以任意改变。
l data)

堆区 一般由程序员分配和释放，若程序员不释放，程序运行结束时由操作系统回
(heap 收。malloc()、calloc ()、free()等函数操作的就是这块内存。也是我们要解
) 释的重点。 注意：这里所说的堆区与数据结构中的堆不是一个概念，堆区的
分配方式倒是类似于链表。空间较大

栈区

(Stack 存放函数的参数值、局部变量的值等，由系统自动管理。空间较小
)

内核区

17、问面试官

①我进入工作中将要面临哪些挑战，我需要在哪些方面精进。

②想了解一下日常的开发流程。

二、证券的外包（一面）

蒋莱科	恒生电子外	孙女	14-17k（期望薪资	证	视频面试：3月4号晚7点
技	包	士	16k)	券	半

1、const关键字修饰

2、GDB调试

(1) 怎么调Core Dump 文件

- `core` :

加载 core dump 文件进行调试。

- 示例: `gdb ./program core` 。

- 结合 `bt` 和 `info` 命令 定位崩溃原因。

(2) 怎么定位段错误

编译时开启调试信息，就是加上 `-g` 选项

运行程序 `gdb + 程序名`, `run`; `gdb` 直接打印错误的代码行

使用`bt`查看调用堆栈

使用 `frame` 切换到崩溃点的栈帧

使用 `info locals` 查看局部变量

使用`print` 查看某个变量

逐步调试: `step (s)`、`next (n)`、`continue (c)`

(3) 怎么调试多线程

使用 `set scheduler-locking on` 锁住其他线程，避免干扰

使用 `info threads` 查看所有线程

使用 `thread + 线程号` 切换到指定线程

(4) 怎么定位出问题的是在哪个线程

`gdb`会自动停到崩溃的线程

使用 `info threads`列出所有线程，标有 `*` 的就是崩溃的线程

使用 frame 0 可以切换到崩溃的栈帧

(5) 检查死锁

使用 thread apply all bt 显示所有线程的调用栈

3、算法了解吗

4、STL的基本常识

(1) 关联式容器

容器名称	描述
vector	动态数组，支持随机访问。
deque /dek/	双端队列，支持两端插入和删除。
list	双向链表，支持高效的插入和删除。list.erase(it); list.rbegin();list.rend();
array (C++11)	定长数组，大小固定。std::array<int, 5> arr2{}; rbegin(); rend(); fill();
forward_list	单向链表，节省空间。

(2) 序列式容器

以键值对的形式存储数据，内部自动排序或提供快速查找。

容器名称	描述
map	有序映射，按键排序。底层是红黑树，查找的时间复杂度是O(log n)
set	有序集合，不允许重复元素。底层是红黑树，查找的时间复杂度是O(log n)
multimap	有序映射，允许重复键。底层是红黑树，查找的时间复杂度是O(log n)
multiset	有序集合，允许重复元素。底层是红黑树，查找的时间复杂度是O(log n)
unordered_map	无序映射，基于哈希表，查找效率更高。底层是哈希表，查找的时间复杂度是O(1)
unordered_set	无序集合，基于哈希表。底层是哈希表，查找的时间复杂度是O(1)

关联式容器

(1) list的基本操作

list 的插入：① `lst.push_back(0);` ② `lst.push_front(1);` ③ `lst.insert(it, 99);`

list 的删除：① `lst.pop_back();` ② `lit.pop_front();` ③ `lst.erase(it);`

list 的合并、排序、去重：① `lst1.merge(list2);` ② `lst.sort();` ③ `lst.unique();` merge前提是已排序的；unique必须先排序。

list 的反转：① `lst.reverse();`

list 的删除满足条件的元素：① `lst.remove_if({ return x%2 == 0;});`

list 的 splice 拼接：① `lst1.splice(lst1.end(), list2);`

(2) vector的基本操作

vector 的访问：① `v[1]` ② `v.at(1)` ③ `v.front();` ④ `v.back();`

vector 的插入和删除：① `v.push_back(1)` ② `v.pop_back();` ③ `v.insert(it, 9);` ④ `v.erase(it);`

修改 vector 的大小：① `v.resize(5, 100);` ② `v.resize(2);` ③ `v.clear();`

查询 vector 的状态：① `v.size();` ② `v.capacity();` ③ `v.empty();`

vector 排序：① `std::sort(v.begin(), v.end());` // 升序排序 ② `std::sort(v.rbegin(), v.rend());` // 降序排序

vector 去重：① `std::sort(v.begin(), v.end());` ② `v.erase(std::unique(v.begin(), v.end()), v.end());`

vector 删除所有满足条件的元素：`v.erase(std::remove_if(v.begin(), v.end(), { return x % 2 == 0; }), v.end());`

(3) deque的基本操作

deque 的访问：① `dq[1]` ② `dq.at(1)` ③ `dq.front();` ④ `dq.back();`

deque 头部的插入和删除：① `dq.push_back(1)` ② `dq.pop_back();` ③ `dq.push_front(1);` ④ `dq.pop_front();`

deque 中间的插入和删除：① dq.insert(it, 9); ② dq.erase(it);

修改 deque 的大小：① dq.resize(5, 100); ② dq.resize(2); ③ dq.clear();

查询 deque 的状态：① dq.size(); ② dq.empty();

deque 排序：① std::sort(dq.begin(), dq.end()); // 升序排序 ② std::sort(dq.rbegin(), dq.rend()); // 降序排序

deque 去重：① std::sort(dq.begin(), dq.end()); (排序) ② dq.erase(std::unique(dq.begin(), dq.end()), dq.end()); (删除)

deque 删除所有满足条件的元素：dq.erase(std::remove_if(dq.begin(), dq.end(), { return x % 2 == 0; }), v.end());

序列式容器

(1) map的基本操作

map 的插入操作：① mp[10] = "hello"; ② mp.insert({3, "hello"}); ③ mp.insert(std::make_pair(3, "hello"));

map 的免拷贝插入：mp.emplace(7, "hello");

map 的删除元素：① mp.erase(3); (删除key = 3的元素) ② mp.erase(it);

map 的查找元素：① auto it = mp.find(2); (返回迭代器) ② mp.count(3); (只会返回 0 或 1)

修改 map：① mp[2] = "Updated"; ② mp.at(1) = "New Value";

获取 map 的状态：① mp.size(); ② mp.empty();

map 的排序：// 按 key 降序排序

```
std::map<int, std::string, std::greater> mp = {{1, "A"}, {3, "C"}, {2, "B"}};
```

(2) set的基本操作

set 的插入操作：st.insert(10);

set 的免拷贝插入：st.emplace(7);

set 的删除元素：① `st.erase(3);`（删除key = 3的元素）② `mp.erase(it);`

set 的查找元素：① `auto it = st.find(2);`（返回迭代器）② `mp.count(3);`（只会返回 0 或 1）

获取 set 的状态：① `st.size();`② `st.empty();`

set 的排序：// 按 key 降序排序
`std::set st = {1, 2, 3};`

set 获取范围内的元素：① `lower_bound(x)`：返回第一个 `>= x` 的元素② `upper_bound(x)`：返回第一个 `> x` 的元素

5、继承

6、构造函数、析构函数

7、Virtual

实现动态绑定，用于实现动态多态

可以用于虚继承（解决菱形继承的问题）

virtual的底层原理：当一个类含有 虚函数，编译器会自动生成一个 虚函数表（VTable），里面存放该类的所有虚函数的 函数指针。

不同继承场景下的虚表数量

总结

场景	虚表数量
无虚函数	0
单继承	1（基类的虚表扩展而来）
多继承	等于基类数量（每个基类贡献一个虚表）
虚继承	每个有虚函数的基类对应一个虚表 + 虚基类管理开销（具体由编译器实现决定）

8、多态

9、MySQL的命令

基本命令

- (1) 查看数据库: `SHOW DATABASES;`
- (2) 创建数据库: `CREATE DATABASE test_db;`
- (3) 切换数据库: `USE test_db;`
- (4) 查看当前数据库: `SELECT DATABASE();`

修改表

- (1) 增加字段: `ALTER TABLE users ADD email VARCHAR(100);`
- (2) 删除字段: `ALTER TABLE users DROP COLUMN email;`

数据操作

- (1) 插入数据: `INSERT INTO users (name, age) VALUES ('Alice', 25);`
- (2) 查询数据: `SELECT * FROM users;`
 - 查询指定字段: `SELECT name, age FROM users;`
 - 带条件查询: `SELECT * FROM users WHERE age > 25;`
 - 排序查询: `SELECT * FROM users ORDER BY age DESC;`
- (3) 删除数据: 不允许删除数据, 只允许修改标记位, 然后再插入新的数据

事务控制

- (1) 开始事务: `START TRANSACTION;`
- (2) 撤销事务: `ROLLBACK;`
- (3) 提交事务: `COMMIT;`

10、内存管理（这个看上面的，主要是下面一个问题）

10、函数的上下文环境

栈：调用的上下文信息（返回地址、参数、局部变量等）。

寄存器：保存部分运行时上下文信息（栈指针、返回地址）。

堆：动态分配的内存（如malloc、new），虽然堆不属于栈帧的一部分，但是它也参与上下文管理。

11、项目中做了哪些工作

三、荣耀相机外包（二面）

中科创达 荣耀外包 侯晴 11-18k（我没给他报） 相机开发 3月4号晚8点半

1、自我介绍

自我介绍的时间在6分钟，还是不错的，显示准备得还是挺充分的

2、做过什么项目

秒传、断点续传、分片传输。构建倒排索引、缓存优化

3、讲一下缓存优化怎么做的

4、线程池的原理

线程池可以提高整体的利用率，CPU的利用率，我不知道你想要哪方面的具体。

线程池是一种预创建并重用线程的技术，主要用于提高并发率，减少线程创建和销毁的开销，并且控制并发量，避免资源被大量线程耗尽

（1）提前创建一组线程，避免频繁创建/销毁线程带来的性能损耗。

（2）任务存在线程队列中，由空闲的线程处理任务。

(3) 任务执行完毕，线程不会销毁，而是返回线程池，等待新的任务。

(4) 线程池还可以控制并发量，防止创建过多的线程。

5、开发过程中怎么用这个线程池可以结合你的业务来讲讲

有线程池，把工作任务放到一个阻塞队列中，然后从线程池当中，然后取走任务，然后再执行任务，这样的话效率比较高，因为有的有长命令。它有长命令和短命令，然后长命令执行的话就占用了线程了，然后其他线程还可以继续别的工作。

6、观察者模式

观察者模式是让观察者和被观察者分离，然后进行解耦，双方互不影响。然后观察者模式有主题是一个纯虚的类，它只定义接口，它不定义具体的实现，然后还有具体主题实现，具体主题实现具体实现，然后有观察者和被观察者，然后他们两个交互是松耦合的，主要用于用于一方变化，之后通知另一方，比如说像微博微博的关注，然后新闻的订阅之类的。

观察者模式用于被观察者状态发生变化时，自动通知所有的观察者，不需要知道观察者的具体信息。

①被观察着保存所有的观察者；②当被观察者状态发生变化时，它会通知所有注册的观察者。③观察者收到通知后，更新自己的状态。

用于消息推送，新闻订阅等。

抽象类

①观察者和被观察者都依赖于虚基类，是松耦合的，只需要维护一个观察者列表，不需要知道它们的具体实现；②一个被观察者可以有多个观察者，便于扩展和修改。

7、文件传输优化

文件传输的话，使用 sendfile系统调用。替代write和替代read和 recv实现零拷贝，用 mmap内存映射，对于大文件的话，然后也有文件秒传，是基于为了避免重复文件的上传，实现了基于文件希值的秒传，每次上传之前都先计算文件的希值，然后再与服务器的进行对比，然后匹配成功就直接上传，然后节约了传输的资源。

①使用 `sendfile` 直接将文件内容从文件描述符传输到套接字描述符，实现零拷贝。②断点续传基于 **HTTP 协议** 和文件的 **分段下载**，通过在请求中指定下载的起始位置和长度，来实现从上次中断的地方继续传输数据。③实现长短命令分离，避免传输大文件的时候占用所有的线程，造成阻塞。④基于文件哈希值实现秒传功能。⑤客户端的超时断开，避免大量占用服务器资源。

8、预防内存泄露？

面试最大的问题，就是回答的太浅了，不能一次性说清楚有关内存泄漏的操作，以及如何预防。

先讲清楚什么叫内存泄漏，再来讲解如何预防

可以使用，可以使用智能指针，然后自动的回收管理，不会就会减少内存泄漏，然后检查的话使用 `Valgrind`等进行检查`memcheck`

①如果自己申请了内存空间，就一定要释放②使用智能指针自动管理内存。避免忘记释放内存。③使用 `shared_ptr` 的时候，要使用`weak_ptr`避免循环引用。③使用 `Valgrind`中的`Memcheck`检测是否有内存泄漏。

9、如何避免死锁

死锁要有4个条件，4个必要条件只要破坏1个，它就不会发生死锁，可以破坏持有并等待，可以让程序一次性的申请所有的资源，但这样的话资源利用率低，然后可以也破坏不可抢占，允许系统强制的回收资源也可以破坏破坏循环等待，让所有的程序都按照固定的顺序申请资源，

方法

固定锁的获取顺序

使用 `std::lock()`

减少锁持有时间

使用 `try_lock()`

避免嵌套锁

引入超时机制

使用 `std::atomic`,替代锁

死锁检测工具

核心思想

避免不同线程按不同顺序获取锁

一次性获取多个锁，避免死锁

只在必要时加锁，尽快释放

失败时不会阻塞，避免长时间等待

不要在锁作用域内调用可能加锁的函数

使用 `wait_for()` 避免无限等待

提高性能，避免死锁

使用`Helgrind` 监测死锁

10、condition条件变量

条件变量的知识很不熟悉，需要加强相关知识的复习。互斥锁解决临界资源的互斥访问，条件变量是实现线程间的同步问题的

`condition_variable` 是 c++ 标准库中的条件变量，允许一个线程等待，直到另一个线程发出通知（`notify_one()` 或 `notify_all()`）。常用于生产者——消费者模型等同步场景。

11、wait调用

Wait会等待互斥量的变化，互斥量变化之后再继续执行程序。

`wait()`让线程进入等待状态，直到收到 `notify_one()` 或 `notify_all()`的通知。

12、notify_one和notify_all

`notify_one()` 唤醒一个等待的线程。

`notify_all()` 唤醒所有等待的线程。

13、lambda表达式

lambda表达式是相当于写一个没有名字的函数，它中括号儿里边儿是捕获列表儿，小括号儿里边儿是参数列表儿，然后后边返回参数，然后最后大括号里边是函数体，它也可以重复使用的话，就需要有一个名字给它定一个名字，它就可以重复使用了。

提供了一种简洁的方式来定义匿名函数。通过 Lambda 表达式，可以在函数内部临时的函数，避免了定义函数的繁琐过程。

```
[捕获列表](参数列表) -> 返回类型 {  
    // 函数体  
}
```

直接使用lambda作为函数

```
#include <iostream>

int main() {
    auto add = [](int a, int b) { return a + b; }; // 定义 Lambda 并存储

    std::cout << add(3, 5) << std::endl; // 输出 8
    std::cout << add(10, 20) << std::endl; // 输出 30

    return 0;
}
```

四、荣耀相机外包三面

1、自我介绍？

包装了3年工作经验？

2、说一下最近负责的工作内容，在里边做了啥？

主要是做的一个ERP管理的项目，然后我在里边主要做了两个模块，一个是传输同步的模块儿，一个是搜索查询的模块。

3、讲一讲在这两个模块具体干了啥？

架构的话是 Reactor的架构，然后主reactor仅用于监听新连接，然后把新连接放到阻塞队列中，然后萨博瑞、艾克特进行创建连接，然后管理旧的连接。管理 Event fd太然后他们进行任务的创建，然后把它交到工作线程的阻塞队列中，然后再把线程池再从阻塞队列中取出任务，然后执行任务。然后我主要做了主要做了里边的断点续传，分片上传，还有在搜索与查询里边是用了一个第三方的库，实现了一个倒排索引。然后是缓存优化。

这里表现得不错了，一个人能说个3分钟左右了

4、断点续传

断点续传回答的也不错，说完后面试官给出的反馈还是挺好的，表示认可了

5、你觉得一般有要入手的话得多长时间？

如果是一些业务逻辑之类的，应该会看一个星期的，看一个星期左右项目，我得先了解一下要做什么。

他跟我说做相机要半年上手

主要看项目的复杂程度，如果很复杂需要的时间肯定会长一些，但我会尽自己最大努力尽快上手 这样回答得不错

6、虚析构造函数的作用 这个问题回答错了

是先定义一个就是定义一个标准的模板，不实现具体的工长由它的派生类定义，定义具体的实现，然后依赖虚机构函数，就可以实现封装，不需要关心具体的实现，可以完成，可以完成解耦，可以完成解耦，在。

当通过基类指针释放派生类对象的时候必须使用虚析构造函数。虚析构造函数的作用是确保派生类的对象被正确析构，防止内存泄露或资源未释放。

7、static关键字的作用

8、位运算

(1) 32位取前16位

先右移16位，与16个1按位与

(2) 32位取后16位

与16个1按位与

(3) 判断一个数是否是2的幂

$n \text{ 与 } (n - 1) == 0;$

(4) 统计一个数的二进制表示中1的个数

$n \text{ 与 } (n - 1) == 0;$

(5) 不使用临时变量，交换两个整数。

$a \wedge= b;$

$b \wedge= a;$

$a \wedge= b;$

(6) 给定一个数组，其中每个元素都出现两次，只有一个元素出现一次，找到这个单一数字。

0与所有成员依次异或

(7) 给定一个32位无符号整数，反转其二进制位。

逐位反转，将最低位移到最高位。

(8) 给定一个包含 0 到 n 的数组，其中缺少一个数字，找到这个缺失的数字。

利用异或运算的性质，将数组中的所有数与 0 到 n 进行异或。

(9) 给定一个数组，其中每个元素都出现两次，只有两个元素出现一次，找到这两个单一数字。

先异或得到两个单一数字的异或结果，然后根据最低位的1分组。

9、STL中选取一个常用的容器，讲一下常用的函数。

10、可不可以接受加班

可以接受加班，只要能学到东西就行。

加班多吗？这个问题没必要问，直接就说现在加班不是很正常嘛，其实都可以接受。

六、酷睿程（智能驾驶）外包

1、自我介绍

好，我叫李博雷，然后我性格偏慢热一点，然后对于代码我态度比较严谨，喜欢深思熟虑之后再输出，然后注重代码的格式，还有可读性，熟悉c++11的相关特性，然后我觉得除了技术之外，团队的配合也非常的重要，不能只顾着研究自己的代码，总体来说比较喜欢c++的开发，希望有机会能够加入到贵公司共同成长，谢谢。

2、讲一下工作经历

我是21年的年底去的广州科维那边工作，然后做的是一个ERP的管理项目。然后里边儿我主要做参与的是两两个模块儿的两个模块儿，一个是传输系统模块儿，一个是搜索查询模块，然后在传输系统当中主要是做了秒传断点续传，分片上传，然后在搜索查询里边主要是做了构建倒排索引，还有做了缓存优化。

3、偏服务端吧

对

4、为什么隔了一年多才参加工作

我正好也赶上疫情了，然后想考研，然后准备了一年多的考研，然后最后最后才去工作的。

5、做的是ERP项目吧

是

6、出现core dump怎么处理

使用GDB

7、具体讲一下怎么使用GDB

要先生成core dump文件，然后运行 GDB，然后程序，然后后边再加上扣的关键字，然后结合BT还有Info的命令定位崩溃的原因，

8、就是你调过吗？比如说你要看某个变量的值什么的

调过，如果调的话要先编译的时候加上杠g的选项，然后查看某一个变量的话是查看所有变量是in for locals，然后也可以用display，然后还有apprent，plan它是单个变量，display是一直显示变量的变化。

9、关于线程同步方面

使用 Condition or variable条件变量是c加标准库提供的条件变量，然后它允许一个线程等待，一直到另一个线程发出通知，然后用wait让线程进入等待状态，直到收到not if one或者是not if all在继续执行。

10、关于锁怎么用

然后可以使用lock，然后可以获取多个锁，然后避免死锁，然后可以使用 Try lock，然后尝试失败的时候就不会阻塞，然后使用wait for，然后可以避免无限的等待，然后使用a Tommy可以避免使用锁，然后能够避免死锁，也有 unique_lock();

11、锁怎么用

Unique_ptr你要先创建一个锁，麦特先创建一个锁，然后接着说。然后锁住它之后，

std::mutex mtx; (定义一个 互斥锁)

unique_lock lock(mtx); (加锁)

作用域结束会自动解锁

也提供了更灵活的锁管理：

(1) defer_lock (延迟加锁)

```
std::mutex mtx;
std::unique_lock<std::mutex> lock(mtx, std::defer_lock); // 先创建，但不加锁
// ... 执行一些逻辑
lock.lock(); // 需要时再加锁
```

(2) try_to_lock (尝试加锁)

```
std::mutex mtx;
std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);
if (lock.owns_lock()) {
    std::cout << "成功获取锁\n";
} else {
    std::cout << "锁被占用\n";
}
```

(3) adopt_lock (接管已有锁)

```
std::mutex mtx;
mtx.lock(); // 手动加锁
std::unique_lock<std::mutex> lock(mtx, std::adopt_lock); // 接管锁
// lock 作用域结束后会自动解锁
```

(4) 手动unlock()

```
std::mutex mtx;

void task() {
    std::unique_lock<std::mutex> lock(mtx);
    std::cout << "任务开始\n";
    lock.unlock(); // 释放锁，让其他线程可以获取锁
    std::cout << "执行不需要锁的代码\n";
}

int main() {
    std::thread t1(task);
    std::thread t2(task);

    t1.join();
    t2.join();
    return 0;
}
```

补充1: **mutex**

mutex.lock(); (手动加锁)

mutex.unlock(); (手动解锁)

补充2: **lock_guard**

作用域结束时，自动释放锁。不能手动释放


```

#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx; // 定义互斥锁

void task(int id) {
    std::lock_guard<std::mutex> lock(mtx); // 加锁
    std::cout << "Thread " << id << " is running\n";
    // lock 作用域结束时自动解锁
}

int main() {
    std::thread t1(task, 1);
    std::thread t2(task, 2);

    t1.join();
    t2.join();
    return 0;
}

```

补充3: `std::lock()`

处理多个互斥锁

```

std::mutex mtx1, mtx2;

void task() {
    std::lock(mtx1, mtx2); // 一次性锁定多个 mutex，避免死锁
    std::lock_guard<std::mutex> lock1(mtx1, std::adopt_lock);
    std::lock_guard<std::mutex> lock2(mtx2, std::adopt_lock);
    std::cout << "多个锁已获取\n";
}

```

12、智能指针

有尤尼克ptr Share的ptr有 vkptr智能指针是通过对象的生命周期来自动管理内存的释放，资源的释放。尤尼和PCI是单独所有权的智能指针，它不能够复制和复制，然后现有的PPI是共享所有权的智能指针使用引用技术来进行管理对象的。

C++ 标准库中的许多组件，如智能指针、文件流、锁等，都是基于 RAII 设计的。

- `auto_ptr` 支持赋值\复制函数,但底层使用的是移交管理权，当我们进行赋值\复制时，原来的`auto_ptr`会被夺取管理权，被置为 `nullptr`。因此被弃用。
- `std::unique_ptr`：独占所有权的智能指针，不能复制 `unique_ptr`，只能移动（通过 `std::move`）。
- `std::shared_ptr`：共享所有权的智能指针，对象的销毁是由引用计数控制的，当最后一个 `shared_ptr` 被销毁时，才会删除对象。适用于多个指针共享资源所有权的情况。
- `std::weak_ptr`：与 `std::shared_ptr` 配合使用，用于避免循环引用

13、unique_ptr 的用法？

不知道

- (1) 使用`make_unique(10)` 来绑定

```

#include <iostream>
#include <memory> // 包含智能指针库

void example() {
    std::unique_ptr<int> ptr = std::make_unique<int>(10); // 分配整数 10
    std::cout << "值: " << *ptr << std::endl; // 访问指针内容
} // 作用域结束，ptr 自动释放内存

int main() {
    example();
    return 0;
}

```

(2) unique_ptr 禁止复制，但是可以移动：使用std::move();

```

std::unique_ptr<int> ptr1 = std::make_unique<int>(20);
// std::unique_ptr<int> ptr2 = ptr1; // ❌ 编译错误（不能复制）

std::unique_ptr<int> ptr2 = std::move(ptr1); // ✅ 允许移动

```

(3) 释放并重新分配资源：reset()

```

std::unique_ptr<int> ptr = std::make_unique<int>(30);
ptr.reset(new int(40)); // 释放 30，并指向 40

```

(4) 释放控制权：release()

```

std::unique_ptr<int> ptr = std::make_unique<int>(50);
int* rawPtr = ptr.release(); // 释放 unique_ptr 控制权，返回裸指针
delete rawPtr; // 需要手动 delete，否则会内存泄漏！

```

(5) unique_ptr作为函数返回值

```

std::unique_ptr<int> createPtr() {
    return std::make_unique<int>(100);
}

int main() {
    std::unique_ptr<int> ptr = createPtr(); // 自动接管返回的资源
    std::cout << *ptr << std::endl;
}

```

(6) unique_ptr可以用于数组

```

std::unique_ptr<int[]> arr = std::make_unique<int[]>(5);
arr[0] = 10;
arr[1] = 20;

```

补充: **unique_ptr**可以做函数参数吗?

不可以

unique_ptr 可以用作函数参数; 但需要使用**std::move()**移交管理权。

14、shared_ptr 的用法

(1) 共享所有权

```

std::shared_ptr<int> p1 = std::make_shared<int>(20);
std::shared_ptr<int> p2 = p1; // 共享所有权

std::cout << "p1 count: " << p1.use_count() << std::endl; // 输出 2
std::cout << "p2 count: " << p2.use_count() << std::endl; // 输出 2

```

(2) reset() 释放资源

```
std::shared_ptr<int> p = std::make_shared<int>(30);  
p.reset(); // 释放资源（如果是最后一个`shared_ptr`）否则就减少引用计数
```

(3) 使用 weak_ptr 避免循环引用

```
struct A;  
struct B;  
  
struct A {  
    std::shared_ptr<B> b_ptr;  
    ~A() { std::cout << "A 析构\n"; }  
};  
  
struct B {  
    std::shared_ptr<A> a_ptr;  
    ~B() { std::cout << "B 析构\n"; }  
};  
  
int main() {  
    auto a = std::make_shared<A>();  
    auto b = std::make_shared<B>();  
    a->b_ptr = b;  
    b->a_ptr = a; // ❌ 形成循环引用  
}
```

修正：

```

struct A {
    std::shared_ptr<B> b_ptr;
    ~A() { std::cout << "A 析构\n"; }
};

struct B {
    std::weak_ptr<A> a_ptr; // 用 `weak_ptr` 代替 `shared_ptr`
    ~B() { std::cout << "B 析构\n"; }
};

```

15、STL用过哪些容器

常用的 Vector，然后历史他 deck，然后还有a map set，an order map，an order set。

16、map和unordered_map的区别

Map的底层是红黑树，它的键是有序的，然后 order map它的是无序的，底层是希表，然后 order map查找效率是 $O(1)$ ，然后map的查找效率是 $\log n$ 。

17、迭代器的失效问题

迭代器失效如果要删除其中的一个元素的话，它可能会造成迭代器失效，它跟 Vector 的长度有关。所以如果要删除一个元素的话，用 it等于然后v点儿 Easier，然后it把这删掉之后，然后让他重新重新指向一下。

迭代器失效统计

操作	vector	list	deque	map/set
扩容（扩展）	全部失效	不失效	全部失效	不失效
插入 (insert)	可能失效	不失效	两端插入部分失效 中间插入全部失效	不失效
删除 (erase)	当前位置及之后失效	仅删除位置失效	删除位置及之前部分失效	仅删除位置失效

迭代器失效解决办法：

(1) 扩容：使用 `reserve()` 重新分配

(2) 插入、删除：重新获取迭代器

18、除了删除还有其他动作会它迭代性失效吗？

不清楚了

19、讲一下单例模式

单例模式是保证全局只有一个全局，只有一个实例，然后再提供一个全局的访问点，对。然后创建的时候会有懒汉式，有饿汉式，懒汉式是只有在需要的时候才会创建，然后饿汉式是程序一旦运行就先创建出来，

20、单例模式的代码

设置成静态的静态的保证只创建一次，然后把虚构造函数删除掉，把赋值函数复制，赋值函数也删除掉，

具体代码 

```
class Singleton {
private:
    static Singleton* instance; // 静态指针
    Singleton() {} // 私有构造函数，防止外部实例化

    Singleton(const Singleton&) = delete; // 删除拷贝构造函数
    Singleton& operator=(const Singleton&) = delete; // 删除拷贝赋值构造函数

public:
    static Singleton* getInstance() {
        if (instance == nullptr) { // 只有在第一次调用时才创建
            instance = new Singleton();
        }
        return instance;
    }
};
```

```
// 初始化静态成员变量
Singleton* Singleton::instance = nullptr;

int main() {
    Singleton* s1 = Singleton::getInstance();
    Singleton* s2 = Singleton::getInstance();
    std::cout << (s1 == s2) << std::endl; // 1, 表明是同一个实例
}
```

删除拷贝构造函数，删除赋值运算符函数，防止通过拷贝创建新的实例。

如果使用了 静态局部变量 或 智能指针管理实例，不需要显示的删除析构函数

21、你的项目里面有一个叫电智通传输和同步系统

传输系统主要是做了秒传，然后断点续传，还有分片上传，然后在里边遇到的困难的话开始做的传输是用瑞说小文件的开始做的时候是小文件的，然后后来一旦上传大文件就会出现出现一些传输速度不稳定，然后CPU占用过高的问题，然后使用就查看日志，然后发现数据会在内核缓冲区，还有用户缓冲区之间多次的拷贝，然后我就使用 style, css、工具，然后 stress工具，然后确认频繁的 read和write，导致用户态和内核态之间多次的拷贝。

然后为了避免使用 read, write让它在用户态和内核态之间多次拷贝，然后用了Linux的 center fail的系统调用，然后实现零拷贝，降低了CPU的占用率。

它可以直接从文件描述符直接传输到网络套件字当中，就避免了用户态和内核态的拷贝。

22、为什么有用户态和内核态的多次的读写。

因为他是一片儿传的，所以他就会从文件是在用户台的，然后它要调用 read或者 write的系统调用它就会切换到内核台，因为文件是它是分片儿传的，所以每一次传输都会从用户态转换到内核态

read 和 write 单次传输大小是2GB，但由于受到①管道的限制，②文件系统块大小的限制，③TCP发送缓冲区大小的限制，④用户缓冲区大小的限制

23、一片多大

2M

24、为什么用了sendfile还要用mmap

map它是它是内存映射，它能够直接将文件映射到对应空间，然后可以避免可以避免要多次拷贝，它也是零拷贝的一种技术

25、sendfile和mmap怎么结合的

客户端传输到服务器，然后接收的时候用IMF然后从服务器传输到从服务器传输到客户端，用的是send file，用的是发送用的send file发送。

26、在公司担任什么样的角色，关于传输同步这一块。

组员、主要做传输方面。

27、服务器部署是谁做的

组长做的

28、你只管写应用，编译成app吗？

不是

29、代码库有了，只管往里填代码吗？

是

30、整个csd这一块是不需要管的

是

31、我问他：具体的开发流程

对具体也就这样，你先把需求分析好了，分析好了，然后觉得这个需求可以做，那就先把方案再出来，做出来之后大家评估好了之后，对吧？详细设计方面对再抽出来，然后再评估一下是ok了，然后大概写代码之前一般都要整理一下你这个模块要做什么怎么样，然后看我给大家提提意见，帮你补充。Ok了之后就开始开发，完了之后，你肯定要在上线，唤醒完了之后，如果有测试人员的话，也要跟告诉他怎么测，

如果没有测试人员，你要先去自测一下，然后把效果说出来，有什么可能会有什么问题，基本上都这样。

七、汉王一面

1、自我介绍一下

2、构造函数有哪些

有拷贝构造、赋值运算符函数

①默认构造函数 ②带参数的构造函数 ③拷贝构造函数 ④移动构造函数

3、怎么定义一个抽象类

1. 包含至少一个纯虚函数（纯虚函数是指函数后面带有 = 0）。
2. 不能直接实例化 抽象类对象。
3. 派生类必须实现所有纯虚函数，否则派生类仍然是抽象类。

```
#include <iostream>
using namespace std;

// 定义抽象类
class Shape {
public:
    // 纯虚函数（没有具体实现）
    virtual void draw() = 0;

    // 可以有普通成员函数
    void info() { cout << "我是一个形状" << endl; }
};

// 继承抽象类并实现纯虚函数
class Circle : public Shape {
```

```

public:
    void draw() override { cout << "画一个圆" << endl; }
};

int main() {
    // Shape shape; // ❌ 错误，不能实例化抽象类
    Circle c;
    c.draw(); // ✅ 画一个圆
    c.info(); // ✅ 我是一个形状
    return 0;
}

```

4、指针和引用的区别

指针是一个存储变量地址的变量，可以指向内存中的某个对象。

引用是变量的别名，必须在定义时初始化，且不能更改指向。

对比项	指针 (Pointer)	引用 (Reference)
是否可变	可以指向不同的对象	绑定后不能更改
是否能为空	可以是 nullptr	不能是 nullptr
是否需要解引用	需要使用 *p	直接使用 ref
是否必须初始化	可以不初始化	必须初始化
内存管理	需要 new/delete	无需手动管理

5、什么是右值引用

右值引用是为了支持临时对象（右值）和资源的转移（而不是复制）而设计的。

右值引用是：&&

左值：一个有持久地址的变量

右值：短生命周期的临时变量

右值引用最主要的作用是 实现移动语义。避免不必要的复制

右值引用也可以用于实现移动赋值操作符，使得对象能够在赋值时进行资源的转移，而不是复制。